



explain how to resolve atlas VCC and 318

Yes, both issues have a clear resolution—but they are different problems:

- VCC is a payment-method and fare-eligibility issue.
- 318 is a duplicate-booking safety response, not a payment failure.

Resolve VCC

1. Check the fare before payment

Before calling `pay.do`, inspect the `offer/verify` response field:

```
supportCreditTransPayment
```

Atlas documents the value as:

- 1: prepayment/deposit only.
- 3: both credit-card passthrough and prepayment are available. ^[1]

If the fare returns 1, do not send a VCC payment. The 403 response is expected because that fare does not support VCC.

2. Select the payment method explicitly

For a VCC-capable fare:

```
paymentMethod = 3  
supportCreditTransPayment = 3
```

For deposit/balance payment:

```
paymentMethod = 1
```

Atlas's documented VCC flow is: confirm that the fare supports VCC, set the VCC payment method, and provide the required card details. The official sandbox also provides test cards for this purpose. ^{[2] [3]}

For your hackathon, I recommend using deposit/balance payment as the primary demo path unless Atlas confirms VCC passthrough is enabled and reliable for your account. VCC is not

necessary to prove Rebound's central value—recovery orchestration.

3. Treat `pay.do` as asynchronous

A successful `pay.do` response only means Atlas accepted the payment request. It does **not** prove that the ticket was issued.

After payment:

```
pay.do
  ↓
queryOrderDetails.do
  ↓
poll until final order/ticket status
```

Check:

- `orderStatus`.
- `ticketStatus`.
- Airline PNR.
- Ticket number.

Atlas specifically recommends `queryOrderDetails.do` as the authoritative follow-up after payment and says to query before retrying if payment may already be processing or complete.

[4]

4. Handle VCC failure safely

If VCC fails:

1. Query the order first.
2. Confirm it is not already ticketed or still processing.
3. Do not blindly retry `pay.do`.
4. Use Atlas's documented hybrid-payment fallback if the order is eligible:
 - Retry with deposit/balance payment, or
 - Regenerate the booking/order context if Atlas requires a new order.
5. Query the new order until the final state.

Important: never expose raw card data in logs, cassettes, model prompts, audit events, or Daytona sandboxes.

For Rebound, the safe product behavior is:

```
VCC attempt fails
→ query authoritative order state
→ if not ticketed and fallback is allowed
→ switch to deposit payment
```

```
→ query until ticketed or failed
→ record the payment method and result in the Recovery Receipt
```

Resolve error 318

Atlas describes 318 as **Duplicate Booking**: the same passenger and flight details already exist in Atlas. Atlas recommends checking whether an existing booking succeeded before ignoring or retrying the failed-looking request. ^[5] ^[6]

Correct handling

When `order.do` returns 318:

1. Do **not** immediately call `order.do` again with the same passenger and flight.
2. Query the existing order using `queryOrderDetails.do`, if you have the original `orderNo`.
3. Check whether it is:
 - Ticketed.
 - Still processing.
 - Paid but awaiting ticketing.
 - Failed or cancelled.
4. If ticketed, return the existing booking instead of creating another one.
5. If processing, wait and poll.
6. Only create a different booking after confirming the original attempt cannot complete and Atlas permits a new booking.
7. Use a genuinely different candidate or wait until the original booking expires when required.

Atlas notes that a new order with identical passenger and flight information can be rejected as a duplicate, and an order cannot simply be amended. ^[7]

What not to do

Do not solve 318 by:

- Changing the passenger name artificially.
- Reusing the same order payload repeatedly.
- Creating fake identities.
- Automatically placing a second booking before querying the first.
- Treating 318 as proof that no booking exists.

Your existing understanding is correct: 318 is a valid business-protection response, not necessarily an Atlas defect.

Rebound implementation

Your executor should model the flow like this:

```
try:
    order = atlas.order(verified_offer)
except AtlasDuplicateBookingError as exc:
    existing = atlas.query_order_details(exc.order_no or original_order_no)

    if existing.is_ticketed:
        return recovered_from_existing_order(existing)

    if existing.is_processing:
        return atlas.poll_order_until_terminal(existing.order_no)

    return mark_attempt_failed("duplicate booking; original order not recoverable")
```

For payment:

```
pay_result = atlas.pay(order, payment_method="deposit")
final_state = atlas.poll_order_until_terminal(order.order_no)

if final_state.is_ticketed:
    return recovered(final_state)

return failed(final_state)
```

The critical design rule is:

Query first, retry second.

That protects against double bookings and duplicate charges.

What to ask Atlas

You should still ask Atlas these focused questions:

“For our hackathon sandbox account, is VCC passthrough enabled for testing, and which exact response field indicates that a specific verified fare supports it?”

“When `order.do` returns 318, what identifier should the client use to query the existing booking, and what exact order/ticket statuses mean it is safe to create a different order?”

“For a VCC failure, can our sandbox account demonstrate the documented fallback from VCC to deposit payment, and should the fallback reuse the existing order or regenerate a new order?”

Bottom line

- Use deposit/balance payment for the reliable hackathon demo.
- Implement VCC only when supportCreditTransPayment confirms eligibility.
- Treat pay.do as an accepted request, not a completed ticket.
- Always use queryOrderDetails.do after payment.
- Treat 318 as “check the existing booking,” never as “retry blindly.”
- For Rebound’s demo, the strongest story is not VCC itself—it is that Rebound avoids duplicate bookings and safely recovers from asynchronous payment or ticketing outcomes.

✱

1. <https://resources.atr iptech.com/api-document/faqs/atlas-api-order>
2. <https://resources.atr iptech.com/api-document/readme/booking-overview/payment-and-ticketing/hybrid-payment-guide>
3. <https://resources.atr iptech.com/api-document/atlas-sandbox/sandbox-test-card>
4. <https://resources.atr iptech.com/api-document/product-guides/booking/booking-step-guides/query-order.md>
5. <https://resources.atr iptech.com/api-document/support-and-reference/troubleshooting-and-support/errors-handling/318-vs-608-duplicate-booking>
6. <https://resources.atr iptech.com/api-document/errors-handling>
7. <https://resources.atr iptech.com/frequently-asked-questions/api-integration-faqs>
8. <https://resources.atr iptech.com/api-document/faqs/atlas-api-payments>
9. <https://resources.atr iptech.com/capability/atlas-news/vcc-pass-through-payments-overview-and-pro-tips>
10. <https://resources.atr iptech.com/api-wen-dang/api-reference/booking-apis/payment-and-ticketing>
11. <https://resources.atr iptech.com/frequently-asked-questions/payments-related-faqs>
12. <https://resources.atr iptech.com/popular-topics/payments/atlas-hybrid-payment-solution>
13. <https://resources.atr iptech.com/>
14. <https://resources.atr iptech.com/chan-pin-jie-shao/atlas-pay>
15. <https://resources.atr iptech.com/api-document/support-and-reference/troubleshooting-and-support/faqs/fulfilment-api-faq>
16. <https://resources.atr iptech.com/api-document>
17. <https://resources.atr iptech.com/api-document/fr-integration>
18. <https://resources.atr iptech.com/capability/atlas-product-overview>
19. <https://resources.atr iptech.com/api-document/atlas-sandbox>
20. <https://resources.atr iptech.com/api-wen-dang/product-guides/booking/booking-step-guides/payment-and-ticketing/hybrid-payment-guide>
21. <https://resources.atr iptech.com/api-document/support-and-reference/troubleshooting-and-support/errors-handling/202-vs-301-vs-308>